

Squeeze Tool Evaluation

Round 2 Report | Bug Fix Verification & Extended Feature Coverage

Prepared following Round 1 (10 tests, 2 critical bugs identified). Round 2 covers regression verification of both critical bugs plus first-time testing of PDF Squeezer, Chat Summarizer, Model Router, and Context Depth Meter.

Tests Run This Round	10 (Tests 11–20)
Critical Bugs Fixed	2 of 2 (dot-collision spacing, keyword corruption)
New Critical Bug Found	1 (PDF Squeezer placeholder-token leak)
Confirmed Unresolved	Polish mode sentence fragments (2nd occurrence)
Newly Tested Features	PDF Squeezer, Chat Summarizer, Model Router, Context Depth Meter
Still Not Observed	Token-savings %, intent-preservation score
Deferred (not built)	Context Vault

Feature Scorecard

Scored out of 10 based on cumulative evidence from Round 1 and Round 2 testing. Detailed rationale for each score follows in the sections below.

Feature	Score /10	Status
Squeeze Mode — Prose Compression	9	Strong, consistent
Code Segmentation (dot-collision + keyword fix)	7	Core bug fixed; new leak found in PDF context
Polish Mode	3	Repeated fragment bug, unresolved
PDF Squeezer	4	Functional but leaks internal tokens into code
Smart Model Router	5	Triggers now, but complexity classification is questionable
Context Transfer / Chat Summarizer	3	Truncates recent text; doesn't synthesize
Context Vault	N/A	Not yet built — deferred
Context Depth Meter	4	Renders, but token count looks understated
Transparency Metrics (savings %, intent score)	1	Never observed across 20 tests
Duplicate Context Stripper	N/A	Untestable in single-shot format — blocked, not scored

Average score across rated features (excluding N/A): **4.8 / 10**

1. Critical Bug Regression Verification

1.1 Dot-Collision Spacing Bug

Status: FIXED

Test 11 confirmed `item.length`, 18.2.0, e.g., `Node.js`, and spread syntax (`[...args]`) all survived untouched when the entire input was code-dense. Test 14 went further, confirming true span-level segmentation: prose compressed normally (“In order to fix” → “to fix”) while code fragments in the same paragraph were left completely intact. This is the correct fix (segment first, then compress prose spans only), not a blunt all-or-nothing bailout.

1.2 Keyword Corruption (“function” → “fn”)

Status: FIXED

Test 13 ran “function” three times in plain prose with no code syntax present, specifically to rule out a code-detection gate as the reason it survived. All three instances were preserved, indicating the term was removed from the abbreviation dictionary at the source, matching the recommended hard-denylist approach rather than a workaround.

2. New Critical Bug Found This Round

2.1 PDF Squeezer — Leaked Internal Placeholder Tokens

Severity: **CRITICAL** — higher severity than either Round 1 bug

- Test 16: a code block inside a real PDF was squeezed. Instead of preserving or corrupting the spread-syntax line, the output contained literal unresolved template artifacts:

Recommended fix: The code-segmentation approach appears to swap code spans out for placeholder tokens (e.g. `__SQUEEZE_CODE_SAFE_7__`) before running prose cleanup, then swap the originals back in afterward. That swap-back step is failing silently in the PDF pipeline, leaking implementation details into user-facing output and producing code that will not run. Add an assertion/validation step after swap-back that fails loudly (and falls back to the original untouched text) if any placeholder token remains in the output, rather than shipping it to the user.

```
const updated = { __SQUEEZE_CODE_SAFE_7__, __SQUEEZE_CODE_SAFE_8__ };
```

This is more severe than either Round 1 bug: those silently altered a value or keyword. This produces code that is visibly broken and reveals internal implementation details of the tool itself.

3. Confirmed Unresolved Issue

3.1 Polish Mode — Broken Sentence Fragments

Severity: **MODERATE** — recurring across two separate tests

- Test 15 (raw Polish-mode test): “thanks” was surgically removed from “thanks so much in advance”, leaving the dangling fragment “so much in advance, really appreciate it!” in the output.
- Test 16 (PDF Squeezer, same failure mode): “It is important to note that” became “It is important to” — an ungrammatical fragment — and the closing paragraph again left “so much in advance for reading thru all of this” dangling after “Thanks” was removed.
- This is the same root cause identified in Round 1: word/phrase-level deletion instead of whole-sentence removal, and it now appears in two independent code paths (Polish mode directly, and prose cleanup inside PDF Squeezer), suggesting the fix was not applied everywhere the relevant rule runs.

Recommended fix: Change the rule from token-level deletion to whole-sentence/clause removal: detect the full conversational-ending clause (e.g. everything from “thanks” through the end punctuation) and remove it as one unit, or leave the whole sentence untouched if it cannot be confidently isolated. Audit every code path that shares this rule — it is clearly invoked from more than one feature.

4. Newly Tested Features

4.1 PDF Squeezer — Overall

Beyond the placeholder-leak bug above, PDF Squeezer performed well on prose: aggressive, coherent compression (“number”→“num”, “through”→“thru”, “with”/“without”→“w”/“w/o”) with intent preserved outside the broken code block and fragment issues. No word count, page count, or compression percentage was surfaced anywhere in the output. A minor formatting artifact was also observed: bullet list items rendered with a stray space before the colon (“Source :” instead of “Source:”).

4.2 Smart Model Router

Test 19 was the first time the Router visibly triggered in the entire evaluation (Rounds 1 and 2 combined). It suggested HAIKU for one prompt and OPUS for another, confirming directional routing logic exists. However the HAIKU suggestion was tagged “LOW TASK — optimized for only 5 words” for a message that contained three explicit numbered sub-questions and a multi-part instruction — a mismatch suggesting the complexity classifier may weight surface features (short sentences, simple vocabulary) more heavily than actual task structure (number of asks, reasoning depth). It is also unclear from the UI whether the suggestion evaluates the assistant's prior message or the user's next input, which should be clarified in the product copy.

4.3 Context Transfer / Chat Summarizer

When tested against this live session, the summarizer output consisted almost entirely of verbatim, truncated user messages with no synthesis of decisions, findings, or bug analysis from the assistant side of the conversation. Truncation also cut off mid-word rather than at a clause or sentence boundary. This suggests the current implementation is closer to a recency-based character-limit truncation than an intent-aware summary, and would likely lose the most valuable content (findings, decisions, next steps) in any real use case.

4.4 Context Depth Meter

The meter does render live, showing “187 / 200K tokens (0%) — SAFE CONTEXT” at a point deep into a long, multi-exchange session that included a full pasted PDF document and multiple detailed exchanges. This figure looks implausibly low for the actual conversation size at that point, suggesting the meter may be counting only the most recent message rather than cumulative thread history — which would cause it to fail to warn users exactly when it matters most (as they approach real context limits). Note: this feature was initially believed to be unbuilt; it was confirmed live and in-product partway through this round, which should be reconciled internally before Round 3.

4.5 Context Vault

Confirmed not yet implemented. Deferred, not scored.

5. Per-Feature Improvement Recommendations

Squeeze Mode (Prose)

- Maintain current behavior — this is the strongest performing feature.
- Consider surfacing the abbreviation choices made (e.g. a toggleable list) so users can override individual substitutions they dislike.

Code Segmentation

- Add automated validation after placeholder swap-back that blocks output containing any unresolved internal token, falling back to original text on failure.
- Extend the hard keyword denylist to cover common false-positive-prone terms beyond 'function' (e.g. 'class', 'return', 'import', 'const') and add regression tests for each.

Polish Mode

- Rebuild the conversational-ending rule to operate on whole clauses/sentences, not individual words.
- Since the same fragment bug appears in both Polish mode and PDF Squeezer's prose cleanup, consolidate this logic into one shared, well-tested function rather than maintaining separate implementations.

PDF Squeezer

- Fix the placeholder leak as top priority — broken code output is a worse user experience than no compression at all.
- Surface word count, page count, and compression percentage in the output, matching the transparency gap noted below.
- Fix the stray space before colons in bullet list formatting.

Smart Model Router

- Re-tune the complexity classifier to weight task structure (number of distinct asks, instructions, or questions) alongside surface-level features like sentence length.
- Clarify in the UI copy whether a suggestion applies to the reply about to be generated or the user's next message.

Chat Summarizer

- Shift from recency-based truncation to genuine synthesis: prioritize decisions, findings, and action items over verbatim recent text.
- If truncation is still used as a fallback, cut at sentence/clause boundaries, never mid-word.

Context Depth Meter

- Audit whether the meter is counting cumulative thread tokens or only the latest message — the current number looks too low for a long session.

- Reconcile build status internally: this appeared live in testing despite being described as not yet implemented.

Transparency Metrics

- Surface token-savings percentage and intent-preservation score on every optimization output — this has now been absent across all 20 tests in both rounds and was an explicit priority recommendation from Round 1.

6. Overall Tool Improvement Roadmap

Looking across both rounds (20 tests total), a few cross-cutting patterns stand out that matter more than any single bug:

Shared logic, shared bugs

The same sentence-fragment failure surfaced independently in Polish mode and PDF Squeezer's prose cleanup. This strongly suggests fragmented or duplicated logic across features rather than one shared, well-tested prose-cleanup module. Consolidating compression rules (spacing, abbreviation, sentence removal, code-safety) into a single shared engine that every feature calls would prevent this class of bug from reappearing feature by feature.

Trust and transparency are still missing

Across all 20 tests, token-savings percentage and intent-preservation score were never shown once. Users currently have no way to verify the tool is doing what it claims, or to quantify the benefit. This is a product-trust issue as much as a feature gap, and should be treated as a release blocker for any public launch, not a nice-to-have.

Silent failure is the recurring failure mode

Dot-collision corruption, keyword corruption, leaked placeholder tokens, and an understated context meter are all cases where the tool produced wrong output with no indication anything had gone wrong. A general design principle would help here: when the tool is uncertain (code detected but not cleanly segmentable, a value that looks like it might be a version number, a token limit approaching), it should visibly flag or degrade gracefully rather than silently guessing.

Newer features are less mature than the core engine

The original Squeeze/prose engine is clearly the most reliable part of the tool (9/10). Everything added around it — Router, Summarizer, Depth Meter, PDF Squeezer — scored in the 3–5 range. Before adding more surface area, it would be worth hardening these existing features against the same rigor the core engine has now received across two rounds of testing.

Build status tracking

Context Depth Meter was believed unbuilt but appeared live mid-round. Worth a quick internal audit of what is actually shipped, in progress, or planned, so evaluation rounds are testing an accurate picture of the product.

Suggested Priority Order for Fixes

- 1 Fix PDF Squeezer placeholder-token leak (broken code shipped to users).
- 2 Consolidate and fix the sentence-fragment removal logic shared by Polish mode and PDF Squeezer.
- 3 Surface token-savings % and intent-preservation score across all optimization outputs.
- 4 Audit and correct Context Depth Meter's token counting.
- 5 Re-tune Model Router's complexity classifier and clarify its UI copy.

- 6 Rebuild Chat Summarizer around synthesis rather than truncation.
- 7 Build out Context Vault (currently deferred).

End of Round 2 Report